Teaching notes: Theorem 1.2

Source: CS1200 Sender-Receiver Exercise 1 reading, theorem on pages 2-3 and proof on pages 3-5.

These notes explain the intended proof and identify corrections needed in the handout. In particular, the printed runtime argument does not establish the theorem's quadratic bound. A corrected allocation strategy is explained below.

1. Begin with what the theorem says.

Say: “A program with a square, two-dimensional memory can be simulated by a program with ordinary, one-dimensional memory. We must preserve its output, whether it halts, and whether it crashes.”

Write down the notation:

Symbol	Meaning
P	Original 2-D program
P′	Simulating 1-D program
n	Number of entries in the original input x
w, $B=2^w$	Original word length and its numerical limit
S	Current side length of the original memory square
S′	Number of allocated cells in the simulator
w′	Simulator's word length, which may be larger
t	Number of steps in a halting, noncrashing original run

The three promises are:

- If the original does not crash, some sufficiently large w′ makes the simulator halt exactly when the original halts, with the same output.
- If the original crashes, the simulator must crash for every w′. A larger simulator word must not hide an original crash.
- A halting original run of t steps has a simulation taking $O((n+t)^2)$ steps for a suitable w′.

Explain why the input is $(B,\max_i x[i],x)$: the simulator needs to know the original word limit, and the maximum input entry lets it check whether any original input value is too large. The first two simulator cells hold this extra information. The simulator keeps the original program's `input_len` equal to n, although its own input has n+2 entries.

2. Show how a grid becomes a list.

Initially use the handout's intended row-by-row representation:

$$M[i][j] \leftrightarrow M'[2+Si+j].$$

Say: “Skip the two information cells, skip i complete rows of length S, then move j cells into the next row.”

For $S=3$:

```
Original grid                Simulator memory

1  2  3                      B | max(x) | 1 2 3 | 4 5 6 | 7 8 9
4  5  6                      indices:    2 3 4   5 6 7   8 9 10
7  8  9
```

Example: $M[1][2]=6$ is stored at index $2+3 \cdot 1+2=7$.

The simulator needs S^2+2 cells. Initially $S=n$, so it allocates enough memory for the $n \times n$ grid. The input is already in the correct positions for row zero; the remaining cells start at zero. This initialization costs $O(n^2)$.

3. Explain the easy instructions.

- **Read or write:** First check that both coordinates satisfy $0 \le i,j<S$. Then use address $2+Si+j$. An invalid access has no effect; an invalid read leaves its destination variable unchanged.
- **Addition and multiplication:** Reproduce the original saturation rule. The simulated results are $\min(a+b,B-1)$ and $\min(ab,B-1)$. For example, when $w=3$, simulated $6+5$ must produce 7, even if the simulator itself can store 11.
- **Other ordinary instructions:** Perform the corresponding operation on the simulated variables.
- **Conditional jumps:** Redirect each jump to the beginning of the block that simulates its original target instruction. A single original instruction may become several simulator instructions.

Say: “Except for growing memory, each original instruction takes only a constant number of simulator operations.”

4. Spend the most time on `MALLOC`.

Original `MALLOC` increases the side length from S to S+1. It creates one new column and one new row, so the number of new cells is

$$(S+1)^2-S^2=2S+1.$$

For growth from 3 to 4, the correct memory contents are:

```
Before:  1 2 3 | 4 5 6 | 7 8 9
After:   1 2 3 0 | 4 5 6 0 | 7 8 9 0 | 0 0 0 0
```

The two information cells are omitted from this picture.

The difficulty is that row lengths change. Every old cell must move from

$$2+Si+j \text{ to } 2+(S+1)i+j.$$

Copy rows from bottom to top and, within each row, from right to left. The destination is never to the left of the source, so this order prevents a write from destroying an old value that has not yet been copied.

A concrete warning: if we copy row 1 before row 2 in the example, row 1 moves into locations formerly holding part of row 2. Copying row 2 first preserves those values.

After copying, explicitly set the new row and new column to zero. Allocating fresh cells does not clear old cells that become gaps between rows. For example, the first new column position previously contained the value 4. The printed pseudocode omits this clearing step.

5. Handle crashes and output explicitly.

The original crashes if an input value or program constant is at least B, or if its memory width exceeds B. Width exactly B is allowed by the definition.

At initialization, check $n>B$, $\max_i x[i] \geq B$, and whether any original assignment constant is at least B. On each simulated `MALLOC`, check whether the new width exceeds B.

When an original crash is required, enter a loop that repeatedly calls the simulator's own `MALLOC`. For every fixed, finite w', that loop eventually exceeds the simulator's memory limit and crashes. It does not merely run forever.

On a successful halt, the output must also be translated. If the original output pointer is p and output length is ℓ, use

$$p'=p+2,\ \ell'=\begin{cases}\min(\ell,S-p), & \text{if } p<S\\ 0, & \text{if } p \geq S\end{cases}$$

Row zero is contiguous in the simulator. Adding 2 skips the information cells; clipping the length prevents the output from spilling into another row or unused memory. The handout does not spell out this final translation.

6. Explain correctness with one invariant.

Say: “After every completed simulation block, the original and simulated programs have matching states.”

More precisely:

- Every original variable has the same simulated value.
- Every valid original memory cell has the same value at its corresponding 1-D address.
- The stored logical width equals the original width.
- The simulator is about to execute the block for the original program's next instruction.

Base case: Initialization puts the input in row zero, initializes the rest of the grid to zero, and sets the simulated variables correctly.

Inductive step: Assuming the states match before an instruction, the read, write, arithmetic, jump, or allocation rule preserves the match afterward. Bounds checks preserve the meaning of invalid accesses; saturation preserves the original arithmetic; moving and clearing cells preserves memory after growth.

Conclusion: For sufficiently large w', every original step has a finite, correct simulation. Halting is preserved, and the translated output is identical. If the original runs forever without crashing, the simulator also runs forever.

Why can one fixed w' suffice even for an infinite run? The original width never exceeds the fixed value B, and original stored values never exceed $B-1$. The simulator's necessary addresses and bookkeeping therefore have a finite bound. We do not need a word length that grows with elapsed time.

7. Flag the runtime gap before claiming the theorem is proved.

The handout correctly observes that $S \leq n+t$: the initial width is n, and each original step can increase it by at most one.

However, its copying routine costs $\Theta(S^2)$ for **each** simulated `MALLOC`. If there are $k \leq t$ allocations, their total cost is

$$\Theta\left(\sum_{r=0}^{k-1} (n+r)^2 \right)$$

With fixed n and k proportional to t, this is $\Theta(t^3)$. A general bound for this construction is $O(n^2+t+t(n+t)^2)$, not the claimed quadratic total.

Say: “Bounding the cost of one allocation does not bound the cost of all allocations. The printed proof needs an additional idea to obtain the theorem's runtime.”

The time spent deliberately forcing a crash is irrelevant to Part 3, because Part 3 concerns runs that halt without crashing.

8. Repair the runtime by reserving extra row width.

This is a modification of the handout's construction.

Keep two separate quantities:

- S: the logical width that the original program is allowed to use.
- C: the reserved row width in the simulator, initially $C=n$.

Reserve C^2+2 cells and use

$$M[i][j] \leftrightarrow M'[2+Ci+j].$$

Always check accesses against S. Reserved cells outside the logical $S \times S$ square stay zero and cannot be accessed by the original program.

On each original `MALLOC`:

1. Increase S and check the original crash condition $S>B$.
2. If $S \le C$, no rearrangement is necessary. The new logical row and column are already zero.
3. If $S>C$, double C, allocate the additional space, copy the old grid backward into the new layout, and clear the new reserved positions. This costs $O(C_{new}^2)$.

Example: starting at width 3, grow the reserved width from 3 to 6 when logical width reaches 4. Logical widths 5 and 6 then require no copying. Grow the reserve to 12 only when logical width reaches 7.

The correctness invariant is the same as before, with stride C replacing stride S, and with the extra condition that unused reserved cells are zero.

Let C_f be the final reserved width. For nonempty input, $C_f<2S_f \le 2(n+t)$. The costs of initialization and all rebuilds form a geometric sum:

$$O(n^2+(2n)^2+(4n)^2+\cdots+C_f^2)$$
$$=O(C_f^2)=O((n+t)^2).$$

Adding $O(t)$ for ordinary instructions and allocations that fit in the reserve leaves the total $O((n+t)^2)$.

Say: “Doubling the reserve makes rebuilds less frequent. Their total cost is within a constant factor of the largest rebuild.”

9. Keep these technical details available for follow-up questions.

Why is a larger word allowed? The simulator addresses roughly the square of the original width. Since $S \le B=2^w$, and the repaired construction has $C<2B$, a word length of $2w+O(1)$ suffices for its memory addresses and bookkeeping. Original data still obey the original $B-1$ saturation limit.

What about Part 2's "every w'" requirement? A small word must cause a crash rather than silently corrupt address calculations. This needs care beyond saying "choose w' large enough." One safe implementation allocates the entire target storage before calculating addresses in it. Allocation counts can be implemented by nested loops over rows and columns, avoiding an overflowing computation of C^2. If physical memory is insufficient, allocation crashes. If allocation succeeds, every valid address in that storage fits in a simulator word. Check the logical crash condition before doubling the reserve. The input B, if it fits at all, ensures enough range for the row and column counters used in this strategy. Implement backward loops with an explicit stop at zero so that unsigned counters do not get stuck there.

What if native addition or multiplication saturates in a small simulator word? If B fits in that word and its maximum value is H, then $H \ge B-1$. Therefore $\min(\min(z,H),B-1)=\min(z,B-1)$. Clamping a native arithmetic result to $B-1$ still reproduces the original data operation correctly. Address calculations must instead remain exact, as ensured by allocating first.

Why is the program fixed while w' may vary? The construction translates the fixed code of P into fixed code for P'. The original word limit comes from input B; choosing a simulator word length changes the machine's storage range, not the translated program.

10. Use this correction checklist while reading the handout.

Location	Correction or missing detail
Page 3, introductory address formula	Use the row width S, not the total 1-D memory size S', as the stride.
Page 3, initial crash check	The definition crashes for $n>B$, not $n \geq B$.
Page 3, reads and writes	A coordinate is invalid when it is $\geq S$, not only when it is $>S$. For a write, check the two address variables, not the value being stored; write into M'.
Page 4, crash after `MALLOC`	Crash when the new logical width is $>B$, not $\geq B$.
Page 4, number of new cells	It is $2S+1$ using the old width, or $2R-1$ using the new width $R=S+1$.
Page 4, copying routine	Explicitly clear newly exposed row and column positions after copying.
Page 4, correctness	Translate the output pointer and clip its length; preserve the original input length; ensure small simulator words cannot silently corrupt bookkeeping.
Page 5, runtime	Initialization costs $O(n^2)$. Sum the costs of all allocations. The printed rebuild-on-every-allocation routine can take cubic time; doubling the reserved stride repairs the bound.

A short script to rehearse

We simulate the 2-D program by storing its rows consecutively in 1-D memory. A cell's address is two plus its row number times the stored row width, plus its column number. The two extra cells record the original word limit and largest input value. We check logical bounds, reproduce saturated arithmetic, and translate jumps. When the grid grows, any rearrangement copies backward and clears new cells. An invariant says the original variables, memory, and next instruction always match the simulated state, so halting and output are preserved. Original crash conditions trigger repeated allocation until the simulator also crashes. The handout's runtime calculation misses the cost of repeated rebuilding. Reserving row width that doubles when full fixes that: rebuild costs form a geometric sum, giving total time $O((n+t)^2)$.

Three checks for your listener

- With stride 4, where is `M[2][1]` stored? Answer: index $2+4 \cdot 2+1=11$.
- If logical width is 4 and reserved width is 6, is column 5 accessible? Answer: no; validity depends on logical width 4.
- Why does $S \leq n+t$ alone fail to prove the runtime bound? Answer: it bounds one rebuild; we still have to add the costs of every rebuild.